

Lab 8 Fall 2026

YouTube Video Statistics Stored in Hash Table

DUE DATES FOR LAB 8 ARE DIFFERENT

There are TWO PARTS to this assignment so the assignment spans over 2 weeks. Part 1 is due next week and Part 2 will be due the week after that. You will submit in two different assignments in Canvas, but your grade for both parts are combined together in a rubric attached to part 2.

CONCEPTS FOR LAB 8

- Hash Tables
- Hash Function with Strings as Keys
- Chaining Collision Management
- Load Factor

DESCRIPTION

A social-media analytics team needs fast lookups of video statistics using video IDs.

You will implement a templated hash table in C++ using an array of linked-list heads (separate chaining).

Your application will load a real, simplified YouTube historical trending-video CSV, maintain one record per video ID, and allow interactive searches, deletions, additions, and updates.

THIS IS NOT A PAIRED PROGRAMMING ASSIGNMENT

All students are to complete this assignment on their own.

You all can talk to each other but you definitely shouldn't have the same code! If you do, that is considered cheating for this assignment (refer to syllabus academic integrity policy).

LAB 8 POLICIES

Generative AI Policy for Lab 8

You are **NOT** allowed to use GitHub Copilot, ChatGPT, Codex, Gemini, or any other AI generation tool for **code generation**. Therefore, if you currently have GitHub Copilot, Codex, or any other GenAI tool enabled in VS Code, you need to **disable it for completing this lab**. You may use Generative AI to aid in **troubleshooting your code** **without uploading the given assignment or source/header files**. You may also not use generative AI in any way to complete part 2 of the assignment (the documentation).

Uploading the assignment or given .cpp or .h files to generate code, fix the given code, or enabling code generation/completion within VSCode will result in an academic integrity violation charge.

What you Can't DO/Use in this lab

If you do any of the following, you will receive a 0 on some or all of the assignment and possibly get charged with an academic integrity violation (this is very serious).

1. You may not use more advanced code or syntax in your lab assignment **that has not been taught** in CSC 1310 class **without referencing your source**. How do you reference your source? By providing a link/URL in a multi-line comment in the section of code where you use the advanced code. **Otherwise, this will be considered plagiarism.** You should be able to explain every line of code that is in your submission.
2. Look at syllabus to review what you may not (ever) use the following in your programming & lab assignments in CSC 1310.
3. You **may not use generative AI (genAI)** to generate code, but you can use it to help in troubleshooting problems as described in the Generative AI Policy above. You **may not use generative AI (genAI)** to answer the part 2 documentation questions.
4. You may not share your code with anyone.

WHAT TO TURN IN TO CANVAS FOR PART 1

Create a **zip** file named **username_lab8_part1.zip** containing the following files and upload it the **Lab 8 Part 1** assignment. Replace username with your TTU. Example: `acrockett43_lab8_part1.zip`

- **CSVHelper.cpp**
- **Driver.cpp**
- **HashTable.h**
- **Video.h**
- **Youtube_hash_lab_data.csv**

WHAT TO TURN IN TO CANVAS FOR PART 2

Lab Report

First, make sure to complete the lab report.

After you have completed part 2 of this assignment, fill out the lab report and submit your lab report in your zip file along with your source files. Name your screenshot **lab8ReportProof**.

Lab Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_0iJcCTCBgguPDpQ

Files to Submit

Create a **zip** file named **username_lab8_part2.zip** containing the following files and upload it the **Lab 8 Part 2** assignment. Replace username with your TTU. Example: `acrockett43_lab8_part2.zip`

I want you to turn in all your files from part 1 in case we need to review them again in order to properly grade your documentation. But then also add your documentation pdf and the report proof.

- **CSVHelper.cpp**
- **Driver.cpp**
- **HashTable.h**
- **Video.h**
- **Youtube_hash_lab_data.csv**
- **Lab8Documentation.pdf**
- **Lab8ReportProof**

DATA SET

The provided data set was first downloaded from **Kaggle** at https://www.kaggle.com/datasets/datasnaek/youtube-new and then heavily modified "cleaned up" to make it usable for this assignment.

Note that many public data sets have missing or incomplete data which could significantly make your code more complex to check for and handle. Therefore, I did that part for you all so that you could focus on the main idea of this assignment, which is hash tables.

However, this data set could contain duplicates.

Dataset reference

Prepared file: **youtube_hash_lab_data.csv**. Header fields and types:

Column	C++ type	Meaning
video_id	string	Unique key for a video
title	string	Video title (may contain commas or quotes)
channel_title	string	Channel name
views	long long	View count
likes	long long	Like count
comment_count	long long	Comment count

The file has 40,552 data rows and 6,281 unique IDs. Some IDs occur repeatedly as a video trended on different dates.

When loading a repeated ID, REPLACE the stored video with the newest row encountered in file order. Do not append duplicate entries for the same key.

The data represents historical trending records, not current YouTube statistics.

PART 1: WRITE THE PROGRAM WITH THE FOLLOWING SPECIFICATIONS

Given Files

All the required files (except the report proof screenshot and documentation document) are already provided for you, but Driver.cpp and HashTable.h are incomplete.

- **CSVHelper.cpp**
- **Driver.cpp**
- **HashTable.h**
- **Video.h**
- **Youtube_hash_lab_data.csv**

HashTable.h

Part of this class is provided for you but it needs to be modified into a template class where the value inside the HashEntry is of the template type instead of just an integer. Keys must remain strings.

Add a comment block at the top of this source file with filename, author, date, and purpose.

You will need to complete all the member functions in this class. Read the comments in the provided header file but also below is additional explanation for some of the functions:

- **indexFor()** – use this member function to implement a polynomial rolling hash function. A polynomial rolling hash function is what you can use when your key is a string and you want to try to distribute keys evenly among the buckets. For more information on this hash function, visit https://cp-algorithms.com/string/string-hashing.html.
  - Start `h=0`
  - For each character `ch` of `key`, calculate `h = (h * 31 + ch) % tablesize`
  - This gives a valid index from 0 through `tableSize-1`, which you will return from this function
  - Do not use std::hash for this lab.

- **HashTable() constructor** – sent an integer size, set tablesize to the size sent tot his function. Set entries to 0. Dynamically allocate the hashArray where each element is a pointer to a HashEntry and the size of the array is in tableSize. Set all array elements to nullptr.
- **~HashTable() destructor** - delete all nodes from all chains, then delete[] the bucket array. Prevent shallow copying (copy constructor and assignment are deleted).
- **putValue()** – parameters include the key (a string) and value (template type); search the ENTIRE selected chain. If the key exists, replace its value and return false; otherwise, allocate one node, add it to the selected chain, increment entries, and return true.
- **getValue()** – parameters include the key (a string) and value (template type); retrieve exact match via the output parameter, returning true; return false if absent.
- **removeValue()** - unlink and delete an entry with provided key from the head, middle, or tail of the linked list at the mapped bucket
- **bucketLength()** – given a particular bucket index, count nodes in this bucket (return -1 for an invalid index).
- **printBucket()** – given a particular bucket index, display the keys in the chain, or report an invalid index.
- **printHashTable()** - display every bucket and its linked list chain.
- **printStatistics()** – print total unique videos, buckets, occupied/collision buckets, longest chain, and average entries per bucket.

Driver.cpp

Add a comment block at the top to indicate the filename, author, date, and purpose.

The main function should first create a Video HashTable with 1010 buckets.

Then, call the already provided loadVideos function, making sure to pass all needed arguments to this function. If loadVideos returns false, you should print that the file couldn't be opened and the program should end.

If the file was able to be opened, there should be a printed statement of how many rows were processed, how many videos were inserted, how many videos were updated, and how many were skipped. (refer to sample output)

Then, a menu should appear with 7 options: find video by ID, add a new video, update video statistics, remove video, display hash table statistics, display a bucket, and exit. Make sure to validate the user's choice before executing the code for that choice.

- For finding a video by ID, ask the user for the id and then call the HashTable class' getValue() function. If found, print the video.
- For adding a new video, ask the user for the video id, reject an already-used ID; get the title, channel, and statistics from user
- For updating video statistics, ask the user for the video id and if it can't be found, tell the user. Otherwise, ask the user for the updated views, likes, and number of comments.
- For removing a video, ask the user for the video id and if it can't be found, tell the user. Otherwise, remove the video and confirm to the user that the video has been removed from the hash table.
- For displaying hash table statistics, refer to sample output to see what should be printed.
- For displaying a bucket, ask the user for the bucket ID and print that bucket's chain of video IDs stored.

You may create as many programmer-defined functions as you want, but they should be defined in this same Driver.cpp file and you must have a comment block above each function to state the function name, purpose of function, and explain what is passed to this function (parameters) and what it returns.

SAMPLE OUTPUT

User input is highlighted in **yellow**.

```
Loaded 40552 rows; 6281 unique videos; 34271 updates; 0 skipped.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
```

```
6. Display a bucket
7. Exit
CHOOSE 1-7: test

You didn't enter in an integer.
CHOOSE 1-7: -3

You didn't enter number 1 through 7.
CHOOSE 1-7: 1


You chose to look up a video.
Enter video ID: 5530I_pYjbo
ID: 5530I_pYjbo
Title: How I Trained My Cats
Channel: JunsKitchen
Views: 3083469
Likes: 193377
Comments: 10440

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 1


You chose to look up a video.
Enter video ID: fxo8CX6rkIk
Video not found.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 2


You chose to add a new video.
Enter new video ID: fxo8CX6rkIk
Title: STAYING MOTIVATED AT SCHOOL with 5 practical tips
Channel: Efficiency and Organization | Ruzanna Hernandez
Views: 239
Likes: 10
Comments: 0
Added.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
```

```
CHOOSE 1-7: 2


You chose to add a new video.
Enter new video ID: fxo8CX6rkIk
That ID already exists; use Update instead.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 3


You chose to update video statistics.
Video ID: fxo8CX6rkIk

Enter updated values.
New views: 245
New likes: 11
New comments: 0
Updated.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 1


You chose to look up a video.
Enter video ID: fxo8CX6rkIk
ID: fxo8CX6rkIk
Title: STAYING MOTIVATED AT SCHOOL with 5 practical tips
Channel: Efficiency and Organization | Ruzanna Hernandez
Views: 245
Likes: 11
Comments: 0

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 5


You chose to print statistics.
Unique videos: 6282
Buckets: 1010
Occupied buckets: 1010
Buckets with collisions: 999
```

```
Longest chain: 16
Average entries per bucket (load factor): 6.2198

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 6


You chose to print a bucket.
Enter bucket index (0 to 1009): 15
Bucket 15: [7iQn-toQp6A] -> [n08glcqdMRY] -> [ijUK3z5rt5s] -> [rYzLh2QuraQ] ->
[GdR0JlvbtWg] -> [76LcOD4aY-o] -> [F1b5ouja_VI] -> [kn4kBibdlWA] -> [Lg5x-YWxpco] ->
[QKwgif8OW1w] -> NULL

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 4


You chose to remove a video.
Video ID: fxo8CX6rkIk
Removed.

========== YOUTUBE VIRAL VIDEO TRACKER ==========
1. Find video by ID
2. Add new video
3. Update video statistics
4. Remove video
5. Display hash table statistics
6. Display a bucket
7. Exit
CHOOSE 1-7: 7
```

PART 2: DOCUMENTATION

Create a document (MS Word or Google Doc) and name it **Lab8Documentation**. Before submitting, save it as **Lab8Documentation.pdf**. Answer the following questions in this document and make sure to put the question number and letter for each response so we can tie it back to the question easily. Points will be deducted if we can't figure out what question you were trying to answer. You can even copy the questions from this document and paste them into your document and make your responses a different color to distinguish your answers froom the questions.

We want reflections from your brain so do not use generative AI to answer these questions.

1. Your hash table currently uses 1,010 buckets to store approximately 6,281 unique videos.
    a. Modify your program temporarily to contain only 5 buckets. Compile & run your code and when you run it, select option 6 to display a bucket. Have your code display bucket index zero. Provide a screenshot of what prints. If all the output won't fit on one screen, its ok to capture part of what is printed.
    [NOTE: make sure to change back to 1010 before submitting].
    b. What happened to the linked lists when only having 5 buckets?
    c. What would happen to the linked lists if you instead used 6,000 buckets?
    d. Would you expect faster searches with 5 or 6000 buckets, and why?
    e. Are there any disadvantages to having more buckets?

2. Examine the indexFor() function in your HashTable.h file.
    a. Explain how this function converts a video ID (which is a string) into an integer representing a bucket index. In your explanation, make sure to include why the function uses the modulo (%) operator.
    b. What could happen if the modulo operation were removed?

3. Two different YouTube video IDs might be assigned to the same bucket.
    a. Explain why this happens even though the video IDs are different.
    b. How does your program handle this situation?

4. Find your putValue() function.
    a. Explain what happens when a new video is added to the hash table.
    b. Explain what happens differently when you attempt to insert a video ID that is already stored in the table.
    c. Why is it important to search the entire linked list before inserting a new node?

5. Suppose you stored all 6,281 unique videos in a regular unsorted array instead of a hash table.
    a. How would searching for a particular video ID be different?
    b. Which data structure would you expect to be more efficient for searching, and why?

6. Your program calculates the average number of entries per bucket (also called the load factor). If your hash table contains 6,281 videos and 1,010 buckets, the load factor is approximately 6.22.
    a. Explain what this number tells you about your hash table.

7. Run your program and select the menu option to display hash table statistics.
    a. Take a screenshot of the number of occupied buckets, the number of buckets with collisions, and the longest chain and place that screenshot in your document.
    b. Based on these results, do you think your hash function is distributing videos reasonably well? Explain your reasoning using at least two statistics from your actual program output.